



Solution Guide: Testing XSS on Web Applications

For this activity, you designed and used malicious JavaScript payloads to test a live production website for stored (persistent) and reflected (non-persistent) cross-site scripting vulnerabilities.

Part 1

- To modify the HTML code with a bold statement, use the following:
 - `<pre>Hello <b>Robert</b></pre>`
- To create a payload that displays an unintended pop-up, use the following:
 - `<script>alert("Hi Robert")</script>`
- To create a payload that displays session cookies, use the following:
 - `<script>alert(document.cookie)</script>`
 - A pop-up will appear with the cookies that might look similar to the following:
 - `PHPSESSID=qqvjspmnr8o47or045tup3sf5; security=low`

Part 2

- To create a payload that displays an unintended pop-up, use the following:
 - `<script>alert("You have been hacked!")</script>`
- To create a payload that displays session cookies, use the following:
 - `<script>alert(document.cookie)</script>`
 - A pop-up will appear with the cookies that might look similar to the following:
 - `PHPSESSID=qqvjspmnr8o47or045tup3sf5; security=low`

Part 3: Mitigation Strategy Questions

1. How could a malicious user take advantage of the vulnerabilities that you just uncovered? What could the potential impacts be?
 - **Reflected XSS:** There are many acceptable answers. If a web application is vulnerable to reflected XSS, various malicious payloads could be sent to victims by using a phishing campaign. Impacts could include the following:
 - Capturing a user's private session cookies if the victim clicks on the phishing XSS payload
 - Loading malware into the user's machine if the victim clicks on the phishing XSS payload
 - **Stored XSS:** There are many acceptable answers. If a web application is vulnerable to stored XSS, various malicious payloads could impact all users who visit the infected page. Impacts could include the following:
 - Capturing a user's private session cookies if the victim accesses the infected webpage
 - Loading malware into the user's machine if the victim accesses the infected webpage
2. How might you mitigate against the reflected and stored vulnerabilities that you just exploited?
 - Answers might include the following:
 - Adding a server-side input validation to deny malicious scripts as inputs
 - Using HTTP response headers to prevent malicious scripts from running
 - Additionally, you can refer to the following resource for a comprehensive list of methods to protect applications from XSS: [OWASP: Cross Site Scripting Prevention Cheat Sheet](#).

Bonus: Circumventing XSS Mitigation Strategies

- The mitigation strategy used on the Medium security level is an input validation code that removes the value of `<script>` or `</script>`.
- However, the input validation code only looks for the lowercase values of `<script>` or `</script>`.

- To bypass this, we can change any or all of the letters in the word `script` to uppercase by using any of the following payloads:
 - `<SCRIPT>alert("Hi Robert")</SCRIPT>`
 - `<Script>alert("Hi Robert")</Script>`
 - `<scripT>alert("Hi Robert")</scripT>`